

Esses exercícios foram gerados pela Gemini

Engenharia Elétrica

1. Cálculo de Queda de Tensão em Linha CC

Enunciado: Um engenheiro precisa calcular a queda de tensão em um cabo de cobre que alimenta uma carga industrial. O algoritmo deve receber a corrente (I), a resistência por metro do cabo (ρ) e a distância (L), retornando se a queda de tensão está dentro do limite de **5%** para uma rede de **220V**.

Pseudocódigo:

Plaintext

None

Algoritmo QuedaTensao

Início

Ler corrente, resistencia_metro, comprimento, tensao_nominal

queda_tensao <- corrente * resistencia_metro * comprimento

percentual <- (queda_tensao / tensao_nominal) * 100

Se percentual <= 5 então

Exibir "Queda Aceitável:", queda_tensao, "V"

Senão

Exibir "Queda Crítica! Redimensione o cabo."

FimSe

Fim

Código Python:

Python

None

```
def calcular_queda_tensao(I, r_m, L, V_nom=220):  
    queda = I * r_m * L  
    perc = (queda / V_nom) * 100  
    status = "Aceitável" if perc <= 5 else "Crítica"  
    return queda, perc, status
```

Exemplo: 15A, 0.005 ohm/m, 50 metros

v, p, s = calcular_queda_tensao(15, 0.005, 50)

print(f"Queda: {v}V ({p:.2f}%) - Status: {s}")

2. Dimensionamento de Banco de Capacitores

Enunciado: Calcule a potência reativa necessária (Q_c) para corrigir o fator de potência (FP) de uma planta industrial de um valor inicial para um valor desejado.

Pseudocódigo:

Plaintext

```
None
Algoritmo CorrecaoFP
Início
    Ler potencia_ativa (P), fp_atual, fp_desejado
    angulo_atual <- arc_cosseno(fp_atual)
    angulo_novo <- arc_cosseno(fp_desejado)
    qc <- P * (tangente(angulo_atual) - tangente(angulo_novo))
    Exibir "Potência do Banco de Capacitores (kVAr):", qc
Fim
```

Código Python:

Python

```
None
import math

def dimensionar_capacitor(P_kw, fp_in, fp_fim):
    phi1 = math.acos(fp_in)
    phi2 = math.acos(fp_fim)
    Qc = P_kw * (math.tan(phi1) - math.tan(phi2))
    return Qc

print(f"Capacitor necessário: {dimensionar_capacitor(100, 0.8, 0.95):.2f} kVAr")
```

3. Análise de Malha Simples (Lei de Ohm)

Enunciado: Crie um programa que receba uma lista de resistências em série e a tensão da fonte, calculando a corrente total e a potência dissipada em cada resistor.

Pseudocódigo:

Plaintext

None

Algoritmo AnaliseSerie

Início

Ler tensao_fonte, lista_resistores

req <- soma(lista_resistores)

itotal <- tensao_fonte / req

Para cada r em lista_resistores faça

potencia <- r * (itotal^2)

Exibir "Resistor", r, "Potência:", potencia

FimPara

Fim

Código Python:

Python

None

```
def analise_serie(V, resistores):
```

```
    req = sum(resistores)
```

```
    i = V / req
```

```
    potencias = [r * (i**2) for r in resistores]
```

```
    return i, potencias
```

```
corrente, p_lista = analise_serie(12, [10, 20, 30])
```

```
print(f"Corrente: {corrente}A | Potências: {p_lista}W")
```

Engenharia Mecânica

4. Ciclo de Vida de Fadiga (Equação de Basquin)

Enunciado: Estime o número de ciclos até a falha (N) de um componente mecânico dada a amplitude de tensão (σ_a) e constantes do material.

Pseudocódigo:

Plaintext

None

Algoritmo FadigaBasquin

Início

Ler amplitude_tensao, constante_a, expoente_b

```
n_ciclos <- (amplitude_tensao / constante_a)^(1 / expoente_b)
Exibir "Vida estimada em ciclos:", n_ciclos
Fim
```

Código Python:

Python

```
None
def estimar_vida_fadiga(sigma_a, a, b):
    # Equação:  $\sigma_a = a * (N^b)$ 
    N = (sigma_a / a)**(1/b)
    return N

# Exemplo material genérico
print(f"Ciclos: {estimar_vida_fadiga(300, 900, -0.1):.0e}")
```

5. Conversão de Torque e Potência em Motores

Enunciado: Converta a rotação (RPM) e o Torque (Nm) de um motor para Potência (kW e HP).

Pseudocódigo:

Plaintext

```
None
Algoritmo PotenciaMotor
Início
    Ler torque, rpm
    omega <- (2 * PI * rpm) / 60
    potencia_watts <- torque * omega
    potencia_kw <- potencia_watts / 1000
    potencia_hp <- potencia_kw * 1.341
    Exibir potencias
Fim
```

Código Python:

Python

None

```
import math

def calcular_potencia_motor(torque, rpm):
    omega = (2 * math.pi * rpm) / 60
    p_kw = (torque * omega) / 1000
    p_hp = p_kw * 1.341
    return p_kw, p_hp

kw, hp = calcular_potencia_motor(250, 3000)
print(f"Potência: {kw:.2f} kW / {hp:.2f} HP")
```

6. Dilatação Térmica Linear

Enunciado: Calcule a folga necessária entre trilhos de trem considerando a variação de temperatura máxima e mínima da região.

Pseudocódigo:

Plaintext

None

```
Algoritmo Dilatacao
Início
    Ler comprimento_inicial, alfa, t_min, t_max
    delta_t <- t_max - t_min
    delta_l <- comprimento_inicial * alfa * delta_t
    Exibir "Folga necessária (m):", delta_l
Fim
```

Código Python:

Python

None

```
def calcular_dilatacao(L0, alfa, t_min, t_max):
    delta_L = L0 * alfa * (t_max - t_min)
    return delta_L

# Aço: 1.2e-5, trilho de 12m, variação de 40 graus
folga = calcular_dilatacao(12, 1.2e-5, 10, 50)
print(f"Folga: {folga*1000:.2f} mm")
```



Engenharia de Telecomunicações

7. Cálculo de Orçamento de Enlace (Link Budget)

Enunciado: Determine a potência recebida em um receptor de rádio considerando a potência do transmissor, ganhos de antena e perdas no espaço livre.

Pseudocódigo:

Plaintext

```
None
Algoritmo LinkBudget
Início
    Ler p_tx_dbm, ganho_tx, ganho_rx, distancia, frequencia
    perda_espaco <- 20*log10(distancia) + 20*log10(frequencia) + 32.44
    p_rx <- p_tx_dbm + ganho_tx + ganho_rx - perda_espaco
    Exibir "Potência Recebida (dBm):", p_rx
Fim
```

Código Python:

Python

```
None
import math

def link_budget(ptx, gtx, grx, d_km, f_mhz):
    # Perda de percurso simplificada (FSPL)
    fspl = 20*math.log10(d_km) + 20*math.log10(f_mhz) + 32.44
    prx = ptx + gtx + grx - fspl
    return prx

print(f"Sinal Recebido: {link_budget(20, 15, 10, 5, 2400):.2f} dBm")
```

8. Conversão Binário para Sinal Unipolar (Codificação de Linha)

Enunciado: Converta uma string binária em uma lista de tensões (0V para bit 0, 5V para bit 1) para simular um sinal digital.

Pseudocódigo:

Plaintext

None

Algoritmo CodificadorUnipolar

Início

Ler string_bits

lista_sinal <- []

Para cada bit em string_bits faça

Se bit == "1" então adicionar 5.0 à lista_sinal

Senão adicionar 0.0 à lista_sinal

FimPara

Exibir lista_sinal

Fim

Código Python:

Python

None

```
def codificar_unipolar(bits):
```

```
    return [5.0 if b == '1' else 0.0 for b in bits]
```

```
sinal = codificar_unipolar("10110")
```

```
print(f"Níveis de tensão: {sinal}")
```

9. Cálculo de Capacidade de Canal (Teorema de Shannon)

Enunciado: Calcule a taxa máxima de transmissão teórica de um canal com largura de banda (B) e relação sinal-ruído (SNR).

Pseudocódigo:

Plaintext

None

Algoritmo CapacidadeShannon

Início

Ler banda_hz, snr_db

snr_linear <- $10^{(snr_db / 10)}$

capacidade <- banda_hz * $\log_2(1 + snr_linear)$

Exibir "Capacidade (bps):", capacidade

Fim

Código Python:

Python

None

```
def capacidade_shannon(B, snr_db):
    snr_linear = 10**(snr_db / 10)
    C = B * math.log2(1 + snr_linear)
    return C

print(f"Capacidade: {capacidade_shannon(1e6, 30)/1e6:.2f} Mbps")
```

Exercício Interdisciplinar (Mix)

10. Sistema de Monitoramento de Falhas

Enunciado: Um sistema monitora a temperatura de um motor (Mecânica), a corrente de alimentação (Elétrica) e envia um alerta via rádio se os limites forem excedidos (Telecom).

Pseudocódigo:

Plaintext

None

```
Algoritmo Monitoramento
Início
    Ler temp_atual, corrente_atual
    limite_temp <- 85
    limite_corrente <- 20
    Se (temp_atual > limite_temp) OU (corrente_atual >
limite_corrente) então
        Exibir "STATUS: ALERTA - Enviando pacote de dados..."
    Senão
        Exibir "STATUS: NORMAL"
    FimSe
Fim
```

Código Python:

Python

None

```
def monitorar_sistema(temp, corrente):
    critico_t = temp > 85
```



```
critico_i = corrente > 20

if critico_t or critico_i:
    msg = f"ALERTA: {'Temp ' if critico_t else ''}{ 'Corrente ' if critico_i else ''}alta!"
else:
    msg = "Sistema Operando Normal."
return msg

print(monitorar_sistema(90, 15))
```

Esses exercícios foram gerados pela ChatGPT

Engenharia Elétrica (4 exercícios)

1. Cálculo de Potência em Circuito Elétrico

Enunciado:

Uma carga elétrica opera com tensão e corrente variáveis ao longo do tempo. Desenvolva um algoritmo que calcule a potência instantânea $P = V \times I$.

Pseudocódigo:

INÍCIO

ler tensão V

ler corrente I

```
P ← V * I
escrever P
FIM
```

Python:

```
V = float(input("Digite a tensão (V): "))
I = float(input("Digite a corrente (A): "))

P = V * I

print(f"Potência: {P} W")
```

2. Cálculo de Energia Consumida

Enunciado:

Calcular a energia consumida por um equipamento ao longo do tempo: $E = P \times t$ $E = P \times t$

Pseudocódigo:

```
INÍCIO
ler potência P
ler tempo t
E ← P * t
escrever E
FIM
```

Python:

```
P = float(input("Potência (W): "))
t = float(input("Tempo (h): "))

E = P * t

print(f"Energia consumida: {E} Wh")
```

3. Lei de Ohm Generalizada

Enunciado:

Dado resistência e corrente, calcule a tensão usando a Lei de Ohm.

Pseudocódigo:

```
INÍCIO
ler resistência R
```

```
ler corrente I
 $V \leftarrow R * I$ 
escrever V
FIM
```

Python:

```
R = float(input("Resistência (Ohms): "))
I = float(input("Corrente (A): "))

V = R * I

print(f"Tensão: {V} V")
```

4. Simulação de Divisor de Tensão

Enunciado:

Calcular a tensão de saída em um divisor de tensão.

Pseudocódigo:

```
INÍCIO
ler Vin, R1, R2
 $V_{out} \leftarrow V_{in} * (R2 / (R1 + R2))$ 
escrever Vout
FIM
```

Python:

```
Vin = float(input("Tensão de entrada: "))
R1 = float(input("R1: "))
R2 = float(input("R2: "))

Vout = Vin * (R2 / (R1 + R2))

print(f"Tensão de saída: {Vout} V")
```

Engenharia Mecânica (3 exercícios)

5. Cálculo de Velocidade Média

Enunciado:

Calcular a velocidade média de um objeto em movimento.

Pseudocódigo:

```
INÍCIO
ler distância d
ler tempo t
 $v \leftarrow d / t$ 
escrever v
FIM
```

Python:

```
d = float(input("Distância (m): "))
t = float(input("Tempo (s): "))

v = d / t

print(f"Velocidade média: {v} m/s")
```

6. Energia Cinética

Enunciado:

Calcular a energia cinética de um corpo.

Pseudocódigo:

```
INÍCIO
ler massa m
ler velocidade v
 $E_c \leftarrow (1/2) * m * v^2$ 
escrever  $E_c$ 
FIM
```

Python:

```
m = float(input("Massa (kg): "))
v = float(input("Velocidade (m/s): "))

 $E_c = 0.5 * m * v^2$ 

print(f"Energia cinética: {E_c} J")
```

7. Cálculo de Força Resultante

Enunciado:

Aplicar a segunda lei de Newton: $F = m \times a$ $F = m \times a$.

Pseudocódigo:

```
INÍCIO  
ler massa m  
ler aceleração a  
 $F \leftarrow m * a$   
escrever F  
FIM
```

Python:

```
m = float(input("Massa (kg): "))  
a = float(input("Aceleração (m/s²): "))  
  
F = m * a  
  
print(f"Força resultante: {F} N")
```



Engenharia de Telecomunicações (3 exercícios)

8. Cálculo de Taxa de Transmissão

Enunciado:

Calcular a taxa de transmissão (bps) com base no número de bits e tempo.

Pseudocódigo:

```
INÍCIO  
ler bits  
ler tempo  
taxa  $\leftarrow$  bits / tempo  
escrever taxa  
FIM
```

Python:

```
bits = float(input("Número de bits: "))  
tempo = float(input("Tempo (s): "))  
  
taxa = bits / tempo  
  
print(f"Taxa de transmissão: {taxa} bps")
```

9. Atenuação de Sinal

Enunciado:

Calcular a perda de sinal em dB.

Pseudocódigo:

```
INÍCIO
ler Pin
ler Pout
atenuacao ← 10 * log10(Pin / Pout)
escrever atenuacao
FIM
```

Python:

```
import math

Pin = float(input("Potência de entrada: "))
Pout = float(input("Potência de saída: "))

atenuacao = 10 * math.log10(Pin / Pout)

print(f"Atenuação: {atenuacao} dB")
```

10. Cálculo de Comprimento de Onda

Enunciado:

Calcular o comprimento de onda de um sinal: $\lambda = c/f$ $\lambda = c / f$

Pseudocódigo:

```
INÍCIO
ler frequência f
c ← 3 * 10^8
lambda ← c / f
escrever lambda
FIM
```

Python:

```
f = float(input("Frequência (Hz): "))
c = 3e8

lambda_ = c / f

print(f"Comprimento de onda: {lambda_} m")
```



Resumo

- ⚡ Engenharia Elétrica: 4 exercícios
- ⚙️ Engenharia Mecânica: 3 exercícios
- 📡 Telecomunicações: 3 exercícios

Se quiser, posso gerar uma versão mais avançada (com vetores, simulações, ou uso de bibliotecas como NumPy).

Desafios de Algoritmos

Engenharia Eletrica · Mecanica · Telecomunicacoes

4 exercicios

Eng. Eletrica

3 exercicios

Eng. Mecanica

3 exercicios

Eng. Telecom

Nivel:

Iniciante

1 Lei de Ohm — calculo de resistencia

[Eng. Eletrica]

ENUNCIADO

Um tecnico precisa dimensionar resistores em um circuito. Dado um conjunto de pares (tensao V,

corrente I), calcule a resistencia $R = V/I$ de cada componente e identifique os que excedem 100 ohms.

PSEUDOCODIGO

ALGORITMO ResistenciaOhm

LEIA lista de pares (V, I)

PARA cada par (V, I) FAÇA

```

SE I != 0 ENTAO
R <- V / I
ESCREVA "R =", R, "ohm"
SE R > 100 ENTAO
ESCREVA "Atencao: resistencia alta!"
FIM SE
SENAO
ESCREVA "Erro: corrente zero (curto-circuito?)"
FIM SE
FIM PARA
FIM ALGORITMO

```

CODIGO PYTHON

```

def calcular_resistencia(pares):
    for v, i in pares:
        if i != 0:
            r = v / i
            print(f"V={v}V, I={i}A -> R = {r:.2f} ohm")
            if r > 100:
                print(" Resistencia acima de 100 ohm")
            else:
                print(f"V={v}V, I=0A -> Erro: corrente zero")
medicoes = [(12, 0.5), (5, 0.0), (220, 1.5), (9, 0.08)]
calcular_resistencia(medicoes)

```

2 Potencia eletrica e consumo em kWh

[Eng. Eletrica]

ENUNCIADO

Uma fabrica monitora o consumo de equipamentos eletricos. Dado a potencia (W) e as horas de uso

diario, calcule o consumo mensal em kWh e o custo estimado (R\$ 0,75/kWh).

PSEUDOCODIGO

ALGORITMO ConsumoEnergia

```

LEIA nome, potencia_W, horas_dia
consumo_dia <- potencia_W * horas_dia / 1000
consumo_mes <- consumo_dia * 30
custo <- consumo_mes * 0.75
ESCREVA "Consumo mensal:", consumo_mes, "kWh"
ESCREVA "Custo estimado: R$", custo
FIM ALGORITMO

```

CODIGO PYTHON

```

def consumo_mensal(equipamentos):
    for nome, potencia_w, horas_dia in equipamentos:
        consumo_kwh = (potencia_w * horas_dia / 1000) * 30
        custo = consumo_kwh * 0.75
        print(f"{nome}: {consumo_kwh:.2f} kWh/mes -> R$ {custo:.2f}")
equipamentos = [
    ("Motor A", 5000, 8),
    ("Iluminacao", 200, 10),
    ("Ar-condicionado", 1500, 6),

```


]

consumo_mensal(equipamentos)

3 Divisor de tensao

[Eng. Eletrica]

ENUNCIADO

Em um circuito divisor de tensao com dois resistores em serie, calcule a tensao na saida

$V_{out} = V_{in} \times R_2 /$

$(R_1 + R_2)$ para diferentes configuracoes e indique qual se aproxima mais de uma tensao alvo.

PSEUDOCODIGO

ALGORITMO DivisorTensao

LEIA V_{in} , V_{alvo} , lista de (R_1 , R_2)

menor_erro <- INFINITO

PARA cada (R_1 , R_2) FAÇA

$V_{out} <- V_{in} \times R_2 / (R_1 + R_2)$

erro <- $|V_{out} - V_{alvo}|$

SE erro < menor_erro ENTAO

menor_erro <- erro

melhor_config <- (R_1 , R_2 , V_{out})

FIM SE

FIM PARA

ESCREVA "Melhor configuracao:", melhor_config

FIM ALGORITMO

CODIGO PYTHON

```
def divisor_tensao(vin, v_alvo, configs):
```

```
    melhor = None
```

```
    menor_erro = float('inf')
```

```
    for r1, r2 in configs:
```

```
        vout = vin * r2 / (r1 + r2)
```

```
        erro = abs(vout - v_alvo)
```

```
        print(f"R1={r1}ohm R2={r2}ohm -> Vout={vout:.2f}V (erro={erro:.2f}V)")
```

```
        if erro < menor_erro:
```

```
            menor_erro = erro
```

```
            melhor = (r1, r2, vout)
```

```
    print(f"Melhor: R1={melhor[0]}ohm, R2={melhor[1]}ohm -> {melhor[2]:.2f}V")
```

```
    configs = [(1000, 1000), (2200, 1000), (4700, 2200)]
```

```
    divisor_tensao(12, 5.0, configs)
```

4 Verificacao de curto-circuito em fusiveis

[Eng. Eletrica]

ENUNCIADO

Um sistema de protecao precisa monitorar fusiveis. Dada uma lista de correntes medidas e a corrente

nominal de cada fusivel, indique quais estao em risco de queimar (corrente > 110% do nominal).

PSEUDOCODIGO

ALGORITMO VerificaFusiveis

LEIA lista de ($i_{fusivel}$, $I_{nominal}$, I_{medida})

PARA cada fusivel FAÇA

```

limite <- I_nominal * 1.10
SE I_medida > limite ENTAO
ESCREVA "ALERTA: Fusivel", id, "em risco!"
SENAO
ESCREVA "Fusivel", id, "OK"
FIM SE
FIM PARA
FIM ALGORITMO
CODIGO PYTHON
def verificar_fusiveis(fusiveis):
for fid, i_nominal, i_medida in fusiveis:
limite = i_nominal * 1.10
status = "ALERTA" if i_medida > limite else "OK"
print(f"Fusivel {fid}: nominal={i_nominal}A, "
f"medido={i_medida}A -> {status}")
fusiveis = [("F1",10,9.5), ("F2",10,11.2), ("F3",20,18.0), ("F4",5,5.6)]
verificar_fusiveis(fusiveis)

```

5 Calculo de tensao mecanica em vigas

[Eng. Mecanica]

ENUNCIADO

Um engenheiro precisa verificar se uma viga suporta uma carga. Dada a forca aplicada (N) e a area da

secao transversal (m2), calcule a tensao $\sigma = F/A$ e verifique se ultrapassa o limite de escoamento do material.

PSEUDOCODIGO

ALGORITMO TensaoMecanica

LEIA lista de (material, F, A, limite_escoamento)

PARA cada viga FACa

$\sigma \leftarrow F / A$

SE $\sigma > \text{limite_escoamento}$ ENTAO

ESCREVA "FALHA: tensao excede limite em", material

SENAO

ESCREVA material, "seguro. Margem:", $\text{limite} - \sigma$, "Pa"

FIM SE

FIM PARA

FIM ALGORITMO

CODIGO PYTHON

```

def tensao_mecanica(vigas):
for material, f, a, limite in vigas:
sigma = f / a
margem = limite - sigma
if sigma > limite:
print(f"{material}: sigma={sigma:.0f} Pa -> FALHA")
else:
print(f"{material}: sigma={sigma:.0f} Pa -> Seguro "
f"(margem={margem:.0f} Pa)")
vigas = [

```

```
("Aco SAE 1020", 50000, 0.002, 210_000_000),  
("Aluminio 6061", 30000, 0.001, 276_000_000),  
("Ferro fundido", 80000, 0.001, 130_000_000),  
]
```

tensao_mecanica(vigas)

6 Conversao de unidades de pressao

[Eng. Mecanica]

ENUNCIADO

Uma linha de producao usa medidores em diferentes unidades (Pa, bar, psi, atm). Crie um conversor que

receba um valor e a unidade de origem e retorne o equivalente em todas as outras unidades.

PSEUDOCODIGO

ALGORITMO ConversorPressao

LEIA valor, unidade_origem

converter tudo para Pa (base)

pressao_pa <- valor * fator[unidade_origem]

PARA cada unidade destino FACA

resultado <- pressao_pa / fator[unidade_destino]

ESCREVA unidade_destino, ":", resultado

FIM PARA

FIM ALGORITMO

CODIGO PYTHON

```
def converter_pressao(valor, unidade):
```

```
fatores = {"Pa": 1, "bar": 1e5, "psi": 6894.76, "atm": 101325}
```

```
if unidade not in fatores:
```

```
    print("Unidade desconhecida")
```

```
    return
```

```
pa = valor * fatores[unidade]
```

```
print(f'{valor} {unidade} equivale a:')
```

```
for u, f in fatores.items():
```

```
    if u != unidade:
```

```
        print(f' {pa/f:.4f} {u}')
```

```
converter_pressao(2.5, "bar")
```

```
converter_pressao(14.7, "psi")
```

7 Calculo de torque e rotacao em motores

[Eng. Mecanica]

ENUNCIADO

Um motor elétrico industrial tem potência (W) e velocidade angular (RPM) variáveis. Calcule o torque $T =$

P / ω (onde $\omega = \text{RPM} \times 2\pi / 60$) e classifique o regime de operação.

PSEUDOCODIGO

ALGORITMO TorqueMotor

LEIA lista de (nome, potencia_W, rpm)

PARA cada motor FACA

$\omega \leftarrow \text{rpm} \times 2 \times \pi / 60$

$\text{torque} \leftarrow \text{potencia_W} / \omega$

SE torque > 100 ENTAO regime <- "pesado"

```
SENAO SE torque > 30 ENTAO regime <- "medio"
SENAO regime <- "leve"
ESCREVA nome, "-> torque:", torque, "Nm -", regime
FIM PARA
```

FIM ALGORITMO

CODIGO PYTHON

```
import math
def calcular_torque(motores):
    for nome, potencia, rpm in motores:
        omega = rpm * 2 * math.pi / 60
        torque = potencia / omega
        if torque > 100:
            regime = "pesado"
        elif torque > 30:
            regime = "medio"
        else:
            regime = "leve"
        print(f"{nome}: T={torque:.2f} Nm -> regime {regime}")
    motores = [
        ("Motor M1", 15000, 1450),
        ("Motor M2", 750, 3000),
        ("Motor M3", 5000, 300),
    ]
    calcular_torque(motores)
```

8 Calculo de atenuacao em cabo coaxial

[Eng. Telecom]

ENUNCIADO

Um tecnico de rede precisa calcular a perda de sinal em cabos coaxiais de diferentes comprimentos.

Dado o coeficiente de atenuacao (dB/m) e o comprimento, calcule se o sinal ainda e utilizavel (acima de -20 dB).

PSEUDOCODIGO

ALGORITMO AtenuacaoCabo

LEIA lista de (trecho, coef_db_m, comprimento_m)

PARA cada trecho FACA

atenuacao <- -1 * coef_db_m * comprimento_m

SE atenuacao > -20 ENTAO

ESCREVA trecho, "OK:", atenuacao, "dB"

SENAO

ESCREVA trecho, "SINAL FRACO:", atenuacao, "dB"

FIM SE

FIM PARA

FIM ALGORITMO

CODIGO PYTHON

```
def atenuacao_cabo(trechos):
    for nome, coef, comprimento in trechos:
        db = -coef * comprimento
```

```

status = "OK" if db > -20 else "SINAL FRACO"
print(f"{nome}: {db:.1f} dB -> {status}")
trechos = [
("Trecho A", 0.05, 150),
("Trecho B", 0.05, 500),
("Trecho C", 0.10, 80),
("Trecho D", 0.02, 1200),
]

```

atenuacao_cabo(trechos)

9 Verificacao de largura de banda de canais

[Eng. Telecom]

ENUNCIADO

Um provedor de internet precisa alocar canais de frequencia. Dado um conjunto de canais com frequencia

inicial e final, verifique sobreposicoes e calcule a largura de banda total disponivel.

PSEUDOCODIGO

ALGORITMO VerificaCanais

LEIA lista de canais (nome, f_inicio, f_fim)

ORDENAR canais por f_inicio

PARA i de 0 ate n-2 FACA

SE canais[i].f_fim > canais[i+1].f_inicio ENTAO

ESCREVA "SOBREPOSICAO entre", canal[i], "e", canal[i+1]

FIM SE

FIM PARA

bw_total <- SOMA(f_fim - f_inicio para cada canal)

ESCREVA "Largura total:", bw_total, "MHz"

FIM ALGORITMO

CODIGO PYTHON

```
def verificar_canais(canais):
```

```
    ordenados = sorted(canais, key=lambda c: c[1])
```

```
    sobreposicao = False
```

```
    for i in range(len(ordenados) - 1):
```

```
        n1, f1i, f1f = ordenados[i]
```

```
        n2, f2i, f2f = ordenados[i+1]
```

```
        if f1f > f2i:
```

```
            print(f"Sobreposicao: {n1} ({f1i}-{f1f}) / {n2} ({f2i}-{f2f})")
```

```
            sobreposicao = True
```

```
    if not sobreposicao:
```

```
        print("Nenhuma sobreposicao detectada.")
```

```
    bw = sum(ff - fi for _, fi, ff in canais)
```

```
    print(f"Largura de banda total: {bw} MHz")
```

```
    canais = [("Canal A", 100, 120), ("Canal B", 115, 135), ("Canal C", 140, 160)]
```

```
    verificar_canais(canais)
```

```
1
```

0 Codificacao binaria de mensagens (ASCII)

[Eng. Telecom]

ENUNCIADO

Um sistema embarcado transmite dados via protocolo serial. Converta uma mensagem de texto para sua representação binária ASCII, exiba byte a byte e calcule o número total de bits transmitidos.

PSEUDOCODIGO

ALGORITMO CodificacaoBinaria

LEIA mensagem (texto)

total_bits <- 0

PARA cada caractere em mensagem FAÇA

codigo_ascii <- ord(caractere)

binario <- converter codigo_ascii para binario (8 bits)

ESCREVA caractere, "->", binario

total_bits <- total_bits + 8

FIM PARA

ESCREVA "Total de bits:", total_bits

FIM ALGORITMO

CODIGO PYTHON

```
def codificar_binario(mensagem):
```

```
    print(f"Codificando: '{mensagem}')
```

```
    for char in mensagem:
```

```
        codigo = ord(char)
```

```
        binario = format(codigo, '08b')
```

```
        print(f"'{char}' -> ASCII {codigo} -> {binario}")
```

```
    total_bits = len(mensagem) * 8
```

```
    print(f"Total de bits transmitidos: {total_bits}")
```

```
    codificar_binario("ENG")
```

Gerado por Claude · Anthropic · Desafios de Algoritmos para Engenharia

Após a análise dos três modelos, foi possível observar diferenças significativas na forma como os exercícios foram gerados.

O ChatGPT apresentou melhor clareza e organização, sendo ideal para aprendizagem inicial. O Gemini destacou-se pela simplicidade, porém com menor profundidade. Já o Claude apresentou soluções mais completas e realistas, com maior complexidade.

Entre os principais problemas encontrados, destacam-se:

- simplificação excessiva em alguns modelos;
- ausência de validação de dados;
- menor contextualização em certos casos.

Dessa forma, o modelo que apresentou melhor desempenho foi o **Claude**, por oferecer maior profundidade técnica, melhor contextualização e soluções mais robustas.